

ENEL445 — Worked Examples: Quadratic Optimisation & Optimality Conditions

David Ewing (82171165) | Covers: variable transformation, optimality condition, Van der Corput, quadratic gradient, exact line search, optimal step size.

Q1. Formulation, Classification and Constrained Optimisation — (a)

[5 marks]

Formulation. You are planning a birthday party for your best friend. You want to spend no more than \$200, while maximising the general enjoyment of the party. The general enjoyment is given by:

$$f(n, r, z) = 30\sqrt{n} + 120\pi r^2 + \frac{10z}{n} \quad (1)$$

where n is the number of people at the party, r is the radius of the birthday cake in metres, and z is the amount spent on snacks and drink. You decide you need to have at least a small ($r \geq 0.2$) cake. The amount of money spent in \$ is given by:

$$c(n, r, z) = 3n + 1000r^2 + z \quad (2)$$

Formulate the optimisation problem in standard form. Identify design variables and include any bounds or constraints implied by the text.

Step 1 — identify the design variables.

Three quantities are free to choose: the number of party guests n , the cake radius r (m), and the snack/drink budget z (\$).

Reason: Every quantity that the decision-maker controls is a design variable. Here n , r , and z all appear in both the objective and the budget, and none is fixed by the problem statement.

Step 2 — state the objective.

The problem asks to *maximise* enjoyment, so the objective is:

$$\max_{n, r, z} f(n, r, z) = 30\sqrt{n} + 120\pi r^2 + \frac{10z}{n}.$$

Reason: Standard form may be stated as either maximisation or minimisation. To convert to minimisation (if required) simply negate: $\min -f$.

Step 3 — extract the constraints from the text.

1. **Budget:** “spend no more than \$200” $\Rightarrow c(n, r, z) \leq 200$, i.e. $3n + 1000r^2 + z \leq 200$.
2. **Minimum cake size:** “at least a small ($r \geq 0.2$) cake” $\Rightarrow r \geq 0.2$.
3. **Physical bounds implied by context:** $n \geq 1$ (need at least one guest); $z \geq 0$ (cannot spend a negative amount on snacks).

Reason: Constraints 1 and 2 are stated explicitly. Constraint 3 is implied: a party with zero guests is meaningless, and a negative snack budget is physically impossible. The question instructs us to include all implied bounds.

Step 4 — write in standard form.

Rewrite the inequality constraints as $g_i \leq 0$ (conventional form):

$$\begin{aligned} g_1(n, r, z) &= 3n + 1000r^2 + z - 200 \leq 0 \quad (\text{budget}) \\ g_2(r) &= 0.2 - r \leq 0 \quad (\text{minimum cake}) \end{aligned}$$

Step 5 — classify the problem.

Reason: Check each component: is the objective linear/nonlinear? Are constraints linear/nonlinear? Are variables continuous or integer?

- **Nonlinear objective:** $\sqrt{n}$ and r^2 are nonlinear; z/n is bilinear (product of reciprocals).
- **Nonlinear constraints:** g_1 contains r^2 , so it is nonlinear.
- **Continuous variables** (treating n as continuous, which is a common modelling assumption unless stated otherwise).

Standard form:

$$\max_{n, r, z} 30\sqrt{n} + 120\pi r^2 + \frac{10z}{n}$$

subject to

$$3n + 1000r^2 + z \leq 200$$

$$r \geq 0.2$$

$$n \geq 1, \quad z \geq 0$$

Classification: constrained, continuous, nonlinear optimisation (three design variables: n, r, z ; two explicit inequality constraints; nonlinear objective and nonlinear budget constraint).

Q1. Formulation, Classification and Constrained Optimisation — (b)

[3 marks]

Convexity. What type of optimisation problem is Q1(a)? Is it (or can it be transformed into) a convex problem?

Step 1 — recall the definition of a convex optimisation problem.

A minimisation problem is convex if the objective function is convex and all inequality constraints are of the form $g_i(x) \leq 0$ with each g_i convex (equivalently, the feasible set is a convex set).

Reason: For a maximisation problem, negate the objective first: $\min -f$. Convexity of $-f$ is then what must be checked.

Step 2 — check convexity of the (negated) objective.

The negated objective is $-f(n, r, z) = -30\sqrt{n} - 120\pi r^2 - \frac{10z}{n}$.

- $-30\sqrt{n}$: the square-root function is concave for $n > 0$, so its negation is **convex**.
- $-120\pi r^2$: a negative quadratic — its Hessian contribution is $-240\pi < 0$, so it is **concave**, not convex.
- $-10z/n$: bilinear in z and $1/n$ — neither convex nor concave in general.

Reason: A sum of functions is convex only if *every* term is convex. The $-r^2$ term is strictly concave, so the sum is not convex.

Step 3 — check convexity of the constraints.

The budget constraint $g_1 = 3n + 1000r^2 + z - 200 \leq 0$ contains $1000r^2$, which is convex in r . The remaining terms are linear. Hence g_1 is convex. The bound $g_2 = 0.2 - r \leq 0$ is linear (convex). So the constraints are convex — but the objective is not.

Step 4 — assess transformability.

- The $-r^2$ term cannot be made convex by a variable substitution without changing the problem structure.
- The bilinear term z/n is not jointly convex in (z, n) .
- No standard convexifying transformation (e.g. log-sum-exp, change of variables) eliminates both issues simultaneously.

Type: Constrained nonlinear optimisation (NLP), non-convex.

Convexity: The problem is **not convex** and cannot readily be transformed into a convex problem. The $-120\pi r^2$ term in the objective (after negation for minimisation) is strictly concave, and the bilinear z/n term is neither convex nor concave. Standard solvers may find local optima only.

Q1. Formulation, Classification and Constrained Optimisation — (c)

[4 marks]

KKT conditions. You decide to simplify by setting $n = 20$, which is the ideal number for the venue. Write down (there is no need to solve) the KKT conditions for the new problem (with n fixed at 20) for r and z .

Step 1 — state the reduced problem.

With $n = 20$ fixed, the objective and budget become:

$$f(r, z) = 30\sqrt{20} + 120\pi r^2 + \frac{10z}{20} = 30\sqrt{20} + 120\pi r^2 + \frac{z}{2}$$

(the constant $30\sqrt{20}$ does not affect the optimum and can be dropped for differentiation purposes).

The reduced problem is:

$$\max_{r, z} 120\pi r^2 + \frac{z}{2}$$

subject to

$$g_1(r, z) = 3(20) + 1000r^2 + z - 200 = 1000r^2 + z - 140 \leq 0$$

$$g_2(r) = 0.2 - r \leq 0$$

$$z \geq 0$$

Reason: Substituting $n = 20$ eliminates one design variable. The constant 60 from 3×20 tightens the budget to $1000r^2 + z \leq 140$.

Step 2 — form the Lagrangian.

Convert to minimisation ($\min -f$) and attach a multiplier $\mu_i \geq 0$ to each inequality constraint $g_i \leq 0$:

$$\mathcal{L}(r, z, \mu_1, \mu_2, \mu_3) = -(120\pi r^2 + \frac{z}{2}) + \mu_1(1000r^2 + z - 140) + \mu_2(0.2 - r) + \mu_3(-z)$$

Reason: KKT multipliers are non-negative for ≤ 0 constraints. The bound $z \geq 0$ is rewritten as $-z \leq 0$ to fit the standard form.

Step 3 — stationarity conditions ($\nabla_x \mathcal{L} = 0$).

$$\frac{\partial \mathcal{L}}{\partial r} = -240\pi r + 2000\mu_1 r - \mu_2 = 0$$

$$\frac{\partial \mathcal{L}}{\partial z} = -\frac{1}{2} + \mu_1 - \mu_3 = 0$$

Step 4 — primal feasibility.

$$1000r^2 + z - 140 \leq 0$$

$$0.2 - r \leq 0$$

$$-z \leq 0$$

Step 5 — dual feasibility.

$$\mu_1 \geq 0, \quad \mu_2 \geq 0, \quad \mu_3 \geq 0$$

Step 6 — complementary slackness.

$$\mu_1 (1000r^2 + z - 140) = 0$$

$$\mu_2 (0.2 - r) = 0$$

$$\mu_3 (-z) = 0$$

Reason: Complementary slackness means: either the multiplier is zero (constraint is inactive and not binding) or the constraint holds with equality (constraint is active). This is what lets us enumerate candidate solutions by checking which constraints are active.

KKT conditions (with $n = 20$, variables r and z):

$$-240\pi r + 2000\mu_1 r - \mu_2 = 0 \quad (\text{stationarity in } r)$$

$$-\frac{1}{2} + \mu_1 - \mu_3 = 0 \quad (\text{stationarity in } z)$$

$$1000r^2 + z \leq 140 \quad (\text{budget})$$

$$r \geq 0.2 \quad (\text{min cake})$$

$$z \geq 0$$

$$\mu_1, \mu_2, \mu_3 \geq 0$$

$$\mu_1(1000r^2 + z - 140) = 0$$

$$\mu_2(0.2 - r) = 0$$

$$\mu_3(-z) = 0$$

Q1. Formulation, Classification and Constrained Optimisation — (d)

[4 marks]

Solve Q1(c). Solve the problem in Q1(c) (with $n = 20$ fixed) by any means. Or: discuss how you would arrive at the solution.

[Maximum 3/4 if the optimal solution is not found.]

Step 1 — recognise that the budget constraint will be active.

The objective $120\pi r^2 + z/2$ increases with both r and z . Any increase in either variable improves the objective, so the budget constraint $1000r^2 + z \leq 140$ will be tight at the optimum (otherwise we could increase z and do better).

Reason: If a constraint is not active at the claimed optimum, we can always improve the objective by relaxing it — contradicting optimality. Hence we set it to equality.

Step 2 — eliminate z using the active budget.

Set $1000r^2 + z = 140 \Rightarrow z = 140 - 1000r^2$.

Substituting into the objective:

$$\begin{aligned} f(r) &= 120\pi r^2 + \frac{1}{2}(140 - 1000r^2) \\ &= 120\pi r^2 + 70 - 500r^2 \\ &= (120\pi - 500)r^2 + 70. \end{aligned}$$

Reason: This is now a one-variable unconstrained problem in r (the constant 70 shifts the value but does not affect the argmax).

Step 3 — determine the sign of the coefficient.

$$120\pi \approx 376.99 \quad \Rightarrow \quad 120\pi - 500 \approx -123.01 < 0.$$

Reason: The coefficient of r^2 is **negative**. This means $f(r)$ is a downward-opening parabola in r — it *decreases* as r increases. To maximise f , we want r as small as possible.

Step 4 — apply the lower bound on r .

The constraint $r \geq 0.2$ gives the smallest feasible value $r^* = 0.2$.

Check $z^* = 140 - 1000(0.2)^2 = 140 - 40 = 100 \geq 0$. ✓

Step 5 — compute the optimal objective value.

$$f^* = (120\pi - 500)(0.04) + 70 = 4.8\pi - 20 + 70 = 4.8\pi + 50 \approx 65.08.$$

Step 6 — verify KKT multipliers are non-negative (spot check).

From stationarity in z : $\mu_1 = \frac{1}{2} > 0$. ✓

From stationarity in r : $\mu_2 = 200 - 48\pi \approx 49.2 > 0$. ✓

$z^* = 100 > 0 \Rightarrow \mu_3 = 0$. ✓

Reason: All multipliers are non-negative and the complementary slackness conditions hold: both g_1 and g_2 are active (multipliers may be positive), and g_3 is inactive so $\mu_3 = 0$. The KKT conditions are satisfied, confirming optimality.

Optimal solution (with $n = 20$):

$$r^* = 0.2 \text{ m}, \quad z^* = \$100, \quad f^* = 4.8\pi + 50 \approx 65.1.$$

The budget constraint and the minimum cake constraint are both active. The key insight is that the net effect of increasing r is *negative* ($120\pi - 500 < 0$), so the cheapest allowable cake is optimal, with all remaining budget going to snacks and drinks.

Q1. Formulation, Classification and Constrained Optimisation — (e)

[4 marks]

Sequential Quadratic Programming (SQP). Briefly explain the main ideas of the SQP algorithm. Ensure you mention how constraints are handled and what considerations are made when performing line search.

Core idea.

SQP solves a constrained nonlinear optimisation problem by solving a sequence of simpler *quadratic programming* (QP) subproblems. At each iteration, the nonlinear objective is approximated by a quadratic model and the nonlinear constraints are linearised, producing a QP that can be solved exactly.

Reason: A quadratic objective with linear constraints is a QP — a well-studied problem class with efficient, reliable solvers. SQP inherits this efficiency by working with QP subproblems rather than the original NLP.

Step 1 — form the QP subproblem at iterate x_k .

The subproblem finds a step direction d by solving:

$$\min_d \frac{1}{2} d^\top B_k d + \nabla f(x_k)^\top d$$

subject to linearised constraints:

$$\nabla g_i(x_k)^\top d + g_i(x_k) \leq 0 \quad \forall i \quad (\text{inequality})$$

$$\nabla h_j(x_k)^\top d + h_j(x_k) = 0 \quad \forall j \quad (\text{equality})$$

where B_k is an approximation to the Hessian of the Lagrangian $\nabla_{xx}^2 \mathcal{L}(x_k, \lambda_k)$.

Reason: The Hessian of the *Lagrangian* (not just the objective) is used because the curvature of the constraints matters at a constrained optimum. In practice B_k is updated quasi-Newton style (e.g. BFGS) to avoid computing second derivatives directly.

Step 2 — handle constraints.

Constraints are handled by *linearisation*: each nonlinear constraint $g_i(x) \leq 0$ is replaced by its first-order Taylor approximation at x_k . The QP subproblem therefore has *linear* constraints, which is what makes it a QP.

- Active constraints are included as equalities in the QP.
- Inactive constraints remain as inequalities.
- The QP solution produces both a step d_k and updated Lagrange multiplier estimates λ_{k+1} .

Reason: Because the constraints are only linearised (not solved exactly), the iterates may temporarily violate the original constraints during intermediate steps — this is called *infeasibility*. Some SQP variants (e.g. feasible-point methods) avoid this by projecting back onto the feasible set after each step.

Step 3 — line search considerations.

Once the direction d_k is found, a step size $\alpha_k > 0$ is chosen so that $x_{k+1} = x_k + \alpha_k d_k$. For constrained problems, standard sufficient-decrease conditions (Armijo, Wolfe) applied to f alone are insufficient — a step that reduces f may violate constraints. Two main approaches are used:

1. **Merit function.** Define a scalar merit function that penalises both objective value and constraint violation, e.g. the ℓ_1 penalty:

$$\phi_\rho(x) = f(x) + \rho \sum_i \max(0, g_i(x)).$$

The step size is chosen to achieve sufficient decrease in ϕ_ρ , balancing optimisation progress against feasibility.

2. **Filter method.** Maintain a “filter” of (objective, infeasibility) pairs and accept a step only if it is not dominated — i.e. it improves either the objective or feasibility (or both) relative to all previous iterates.

Reason: The penalty parameter ρ in the merit function must be chosen large enough that the correct optimum is not masked by the penalty, but not so large that the line search becomes ill-conditioned. Filter methods avoid this tuning issue at the cost of more bookkeeping.

SQP in summary:

1. At x_k , form a QP by quadratically approximating the Lagrangian and linearising all constraints.
2. Solve the QP to get step direction d_k and multiplier estimates λ_{k+1} .
3. Perform a line search using a merit function (or filter) to choose step size α_k that balances objective reduction and constraint feasibility.
4. Update $x_{k+1} = x_k + \alpha_k d_k$; update B_k via BFGS.
5. Repeat until $\|d_k\| \approx 0$ and KKT conditions are satisfied to tolerance.

Constraints are handled by linearisation inside the QP. Line search must use a merit function or filter — not just the objective — to account for constraint satisfaction.

Q2. Unconstrained Optimisation (1) — (a)

[5 marks]

Design variable transformation. Suppose that the i -th design variable of an optimisation problem, x_i , must satisfy $|x_i| \leq a$. Show how to eliminate this constraint by applying variable transformation based on the sigmoid function $r = \frac{1}{1 + \exp(-t)}$.

Step 1 — restate the constraint.

$|x_i| \leq a$ is equivalent to $x_i \in [-a, a]$, an interval of width $2a$.

$$-a < x_i < +a$$

Step 2 — range of the sigmoid.

$$\sigma(t) = \frac{1}{1 + e^{-t}}, \quad t \in \mathbb{R} \Rightarrow \sigma(t) \in (0, 1).$$

Reason: If $t \rightarrow +\infty$, $e^{-t} \rightarrow 0$ so $\sigma \rightarrow 1$. If $t \rightarrow -\infty$, $e^{-t} \rightarrow \infty$ so $\sigma \rightarrow 0$. The image is strictly $(0, 1)$ for all finite t .

Step 3 — scale and shift to $(-a, a)$.

Multiply by $2a$ to obtain range $(0, 2a)$, then shift by $-a$:

$$\begin{aligned} 2a\sigma(t_i) &\in (0, 2a) \\ x_i = -a + 2a\sigma(t_i) &\in (-a, +a). \end{aligned}$$

Reason: Sanity check: $t_i \rightarrow -\infty \Rightarrow x_i = -a + 0 = -a$. $t_i \rightarrow +\infty \Rightarrow x_i = -a + 2a = +a$. For all finite t_i , $x_i \in (-a, +a) \subseteq [-a, a]$. ✓

Step 4 — explicit bound confirmation.

For all finite $t_i \in \mathbb{R}$, the transformed variable satisfies:

$$x_i \in (-a, +a).$$

The open interval $(-a, +a)$ is **always** a subset of the closed constraint set:

$$(-a, +a) \subseteq [-a, a]. \checkmark$$

Reason: The sigmoid never reaches its limits for finite t_i , so the image is strictly the open interval $(-a, +a)$, giving margin on both sides. The constraint is therefore satisfied automatically and can be dropped entirely from the optimisation.

Why use $\sigma(t)$?

The sigmoid is one member of a family of functions that share three key properties required for this variable-transformation trick:

1. **Bounded range** — maps $\mathbb{R}$ into a finite interval, so the constraint on x_i is encoded implicitly.
2. **Smooth and differentiable everywhere** — so the chain rule $\partial f / \partial t_i = (\partial f / \partial x_i)(\partial x_i / \partial t_i)$ is well defined, and gradient-based optimisers can be used on t_i without modification.
3. **Strictly monotone** — the mapping $t_i \mapsto x_i$ is one-to-one, so no information is lost and every $x_i \in (-a, a)$ has a unique pre-image.

Reason: Any smooth, strictly monotone function with bounded range works. Alternatives include $\tanh(t)$ (range $(-1, 1)$, scaled by a) and $\frac{2}{\pi} \arctan(t)$ (range $(-1, 1)$). The sigmoid is convenient because its range is exactly $(0, 1)$, which maps cleanly to $(-a, a)$ with one multiply-and-shift. Without this trick, a constrained method (e.g. KKT conditions) would be needed; the transformation converts the problem into an unconstrained one solvable by any standard solver.

Substitute $x_i = -a + \frac{2a}{1 + e^{-t_i}}$ and optimise over unconstrained $t_i \in \mathbb{R}$. The constraint $|x_i| \leq a$ is then automatically satisfied and can be dropped.

Equivalent forms: $x_i = a \left(\frac{2}{1 + e^{-t_i}} - 1 \right) = a \tanh\left(\frac{t_i}{2}\right)$.

Q2. Unconstrained Optimisation (1) — (b)

[5 marks]

First-order necessary optimality condition. Show that if x^* is a local minimum of the objective function $f(x)$, we must have $\nabla f(x^*) = 0$. *Hint: use the first-order Taylor-series expansion of $f(x)$.*

Step 1 — local-minimum definition. There exists $\delta > 0$ such that

$$f(x^* + d) \geq f(x^*) \quad \text{for all } \|d\| < \delta.$$

Step 2 — first-order Taylor expansion about x^* .

$$\begin{aligned} f(x) &= f(x^*) + \nabla f(x^*)^\top (x - x^*) + O(\|x - x^*\|^2) \\ f(x^* + d) &= f(x^*) + \nabla f(x^*)^\top d + O(\|d\|^2). \end{aligned}$$

Reason: The chain rule gives the linear term $\nabla f(x^*)^\top d$; higher-order terms are $O(\|d\|^2)$.

Step 3 — substitute into the local-minimum inequality.

Substituting the Taylor expansion into $f(x^* + d) \geq f(x^*)$:

$$\begin{aligned} f(x^*) + \nabla f(x^*)^\top d + O(\|d\|^2) &\geq f(x^*) \\ \nabla f(x^*)^\top d + O(\|d\|^2) &\geq 0 \quad \text{for all } \|d\| < \delta. \end{aligned}$$

Step 4 — choose a test direction (contradiction). Set $d = -\varepsilon \nabla f(x^*)$ with $\varepsilon > 0$ small:

$$-\varepsilon \|\nabla f(x^*)\|^2 + O(\varepsilon^2) \geq 0.$$

Step 5 — divide by ε and let $\varepsilon \rightarrow 0^+$.

Reason: Dividing by $\varepsilon > 0$ preserves the inequality; the $O(\varepsilon)$ term vanishes.

$$\begin{aligned} -\|\nabla f(x^*)\|^2 &\geq 0 \\ \|\nabla f(x^*)\|^2 &\leq 0 \end{aligned}$$

But a squared norm is **always** non-negative:

$$\|\nabla f(x^*)\|^2 \geq 0$$

$$0 \leq \|\nabla f(x^*)\|^2 \leq 0 \implies \|\nabla f(x^*)\|^2 = 0 \implies \nabla f(x^*) = 0. \square$$

Q2. Unconstrained Optimisation (1) — (c)

[10 marks]

Consider generating a 5-element *van der Corput* sequence of base b to realise the multi-start initialisation strategy to solve an optimisation problem.

For this purpose, we first represent $i = 1, 2, \dots, 5$ as a $(r+1)$ -digit $(a_r a_{r-1} \dots a_0)_b$ in base b such that $i = a_0 + a_1 b + a_2 b^2 + \dots + a_r b^r$.

Next, find the i -th element of the *van der Corput* sequence, $\phi_i(b)$, using

$$\phi_i(b) = \frac{a_0}{b} + \frac{a_1}{b^2} + \frac{a_2}{b^3} + \dots + \frac{a_r}{b^{r+1}}.$$

Compute $\phi_i(3)$ for $i = 1, 2, \dots, 5$. Show all your work below.

Step 1 — write each i in base 3 and read off digits $a_0, a_1, \dots$

For base $b = 3$, divide i repeatedly by 3 and record remainders (least significant first):

1. Write i in base $b = 3$, reading off digits $a_0, a_1, \dots, a_r$ least-significant first.
2. Each digit a_k contributes a_k/b^{k+1} to the sum.
3. Sum all contributions: $\phi_i(3) = \sum_{k=0}^r \frac{a_k}{3^{k+1}}$.

Step 2 — compute $\phi_i(3)$ for each i .

i	Base-3 repr.	Digits $a_0, a_1, \dots$	Computation	$\phi_i(3)$
1	$(1)_3$	$a_0 = 1$	$1/3$	$1/3$
2	$(2)_3$	$a_0 = 2$	$2/3$	$2/3$
3	$(10)_3$	$a_0 = 0, a_1 = 1$	$0/3 + 1/9$	$1/9$
4	$(11)_3$	$a_0 = 1, a_1 = 1$	$1/3 + 1/9$	$4/9$
5	$(12)_3$	$a_0 = 2, a_1 = 1$	$2/3 + 1/9$	$7/9$

Reason: $i = 1$: $(1)_3, a_0 = 1 \Rightarrow 1/3$. $i = 2$: $(2)_3, a_0 = 2 \Rightarrow 2/3$.

$i = 3$: $3 = (1 \cdot 3 + 0) = (10)_3, a_0 = 0, a_1 = 1 \Rightarrow 0/3 + 1/9 = 1/9$.

$i = 4$: $4 = (1 \cdot 3 + 1) = (11)_3, a_0 = 1, a_1 = 1 \Rightarrow 1/3 + 1/9 = 4/9$.

$i = 5$: $5 = (1 \cdot 3 + 2) = (12)_3, a_0 = 2, a_1 = 1 \Rightarrow 2/3 + 1/9 = 7/9$.

Van der Corput sequence in base $b = 3$ for $i = 1, \dots, 5$:

$$\phi_1(3) = \frac{1}{3}$$

$$\phi_2(3) = \frac{2}{3}$$

$$\phi_3(3) = \frac{1}{9}$$

$$\phi_4(3) = \frac{4}{9}$$

$$\phi_5(3) = \frac{7}{9}.$$

Q3. Unconstrained Optimisation (2) — (i)

[3 marks]

Suppose that the objective function of a convex unconstrained optimisation problem is quadratic with respect to the design variables. That is, $f(x) = \frac{1}{2}x^\top Ax + b^\top x$, where A is a positive definite matrix.

Compute the gradient of $f(x)$ at $x = c$ (i.e., find $\nabla f(c)$).

Hint: you may want to use the result $\frac{\partial a^\top x}{\partial x} = a$ and the fact that A is symmetric due to its positive definiteness.

Step 1 — differentiate the linear term.

Using the standard result $\partial(a^\top x)/\partial x = a$:

$$\frac{\partial}{\partial x}(b^\top x) = b.$$

Reason: Expand: $b^\top x = \sum_i b_i x_i$. The j -th partial is $\partial(\sum_i b_i x_i)/\partial x_j = b_j$, giving the vector b .

Step 2 — differentiate the quadratic term.

Apply the matrix product rule to $\frac{1}{2}x^\top Ax$, then use $A = A^\top$:

$$\begin{aligned}\frac{\partial}{\partial x}\left(\frac{1}{2}x^\top Ax\right) &= \frac{1}{2}(A + A^\top)x \\ &= \frac{1}{2}(A + A)x \quad (A = A^\top \text{ since } A \succ 0) \\ &= Ax.\end{aligned}$$

Reason: The product rule for $x^\top Ax$ gives $\frac{1}{2}(A + A^\top)x$. Since A is positive definite it is symmetric ($A^\top = A$), so $(A + A^\top) = 2A$, collapsing to Ax .

Step 3 — combine and evaluate at $x = c$.

$$\begin{aligned}\nabla f(x) &= Ax + b, \\ \nabla f(c) &= Ac + b.\end{aligned}$$

$$\nabla f(c) = Ac + b.$$

Q3. Unconstrained Optimisation (2) — (ii)

[5 marks]

Line search: We would like to perform an exact line search at c along the direction $-\nabla f(c)$. Formulate the exact line search problem as an unconstrained optimisation problem with the step size μ as the design variable.

$$\min_{\mu \geq 0} \phi(\mu) = f(c - \mu \nabla f(c)).$$

Reason: We are at the current point c and search along the steepest-descent direction $-\nabla f(c)$. The scalar $\mu \geq 0$ controls how far we step. For a quadratic with $A \succ 0$, $\phi(\mu)$ is convex with a unique minimiser $\mu^* > 0$, so the constraint is automatically satisfied.

Q3. Unconstrained Optimisation (2) — (iii)

[12 marks]

Optimal step size: Solve the unconstrained optimisation problem you formulated in the previous question for the optimal step size μ^* .

Write out the updated solution $d = c - \mu^* \nabla f(c)$.

Let $g = \nabla f(c) = Ac + b$ (from Q3(i)).

Step 1 — differentiate $\phi(\mu)$ via chain rule:

$$\begin{aligned}\frac{d\phi}{d\mu} &= (\nabla f(c - \mu g))^{\top} \cdot \frac{d(c - \mu g)}{d\mu} \\ &= (\nabla f(c - \mu g))^{\top} \cdot (-g) \\ &= -\nabla f(c - \mu g)^{\top} g \\ &= 0 \quad (\text{set to zero for optimality}).\end{aligned}$$

Reason: Chain rule: derivative of the outer function (∇f evaluated at $c - \mu g$) times derivative of the inner function $(-g)$.

Step 2 — evaluate $\nabla f(c - \mu g)$ using the result from Q3(i):

Apply $\nabla f(x) = Ax + b$ at $x = c - \mu g$:

$$\begin{aligned}\nabla f(c - \mu g) &= A(c - \mu g) + b \\ &= Ac - \mu Ag + b \\ &= (Ac + b) - \mu Ag \\ &= g - \mu Ag.\end{aligned}$$

Step 3 — set to zero:

$$\begin{aligned}-(g - \mu Ag)^{\top} g &= 0 \\ -g^{\top} g + \mu g^{\top} Ag &= 0 \\ \mu^* &= \frac{g^{\top} g}{g^{\top} Ag}.\end{aligned}$$

Reason: Expand: $-g^{\top} g + \mu g^{\top} Ag = 0$. Since $A \succ 0$, $g^{\top} Ag > 0$, so we can divide through.

$$\mu^* = \frac{\|\nabla f(c)\|^2}{\nabla f(c)^{\top} A \nabla f(c)}, \quad d = c - \frac{\|\nabla f(c)\|^2}{\nabla f(c)^{\top} A \nabla f(c)} \nabla f(c).$$

This is the exact steepest-descent step size for a quadratic objective.

Q4. Optimisation-based Control — (a)

[8 marks]

Consider the system $x_{k+1} = u_k$ with stage cost $c(x_k, u_k)$:

	$u = 1$	$u = 2$	$u = 3$
$x = 1$	4	3	2
$x = 2$	1	2	3
$x = 3$	3	1	2

and terminal cost $J_h(x_h) = x_h$. Find the optimal u_{h-1} and u_{h-2} for any initial value $x_{h-2} \in \{1, 2, 3\}$ recursively using dynamic programming.

Bellman recursion:

$$J_k(x_k) = \min_{u_k} [c(x_k, u_k) + J_{k+1}(x_{k+1})], \quad x_{k+1} = u_k.$$

Step 1 — stage $h - 1$ (one step to go):

Substitute the terminal cost $J_h(u) = u$:

$$\begin{aligned} J_{h-1}(x_{h-1}) &= \min_u [c(x_{h-1}, u) + J_h(u)] \\ &= \min_u [c(x_{h-1}, u) + u]. \end{aligned}$$

x_{h-1}	$u = 1$	$u = 2$	$u = 3$	J_{h-1}	u_{h-1}^*
1	4 + 1 = 5	3 + 2 = 5	2 + 3 = 5	5	1, 2, 3 (tie)
2	1 + 1 = 2	2 + 2 = 4	3 + 3 = 6	2	1
3	3 + 1 = 4	1 + 2 = 3	2 + 3 = 5	3	2

Reason: $J_h(u) = u$ (terminal cost), so the look-ahead cost equals the control value itself. For $x = 1$ all three actions tie at 5; convention selects $u^* = 1$.

Step 2 — stage $h - 2$ (two steps to go):

$$J_{h-2}(x_{h-2}) = \min_u [c(x_{h-2}, u) + J_{h-1}(u)].$$

Using $J_{h-1}(1) = 5$, $J_{h-1}(2) = 2$, $J_{h-1}(3) = 3$:

x_{h-2}	$u = 1 : +5$	$u = 2 : +2$	$u = 3 : +3$	J_{h-2}	u_{h-2}^*
1	4 + 5 = 9	3 + 2 = 5	2 + 3 = 5	5	2 (or 3)
2	1 + 5 = 6	2 + 2 = 4	3 + 3 = 6	4	2
3	3 + 5 = 8	1 + 2 = 3	2 + 3 = 5	3	2

Reason: The dominant action at stage $h - 2$ is $u_{h-2}^* = 2$ for all starting states because row $x = 2$ of the cost table has the lowest stage cost and $J_{h-1}(2) = 2$ is the cheapest future cost.

Optimal two-step policy:

$$u_{h-2}^* = 2 \quad \text{for all } x_{h-2} \in \{1, 2, 3\}.$$

This drives $x_{h-1} = 2$, giving $u_{h-1}^* = 1$ and terminal state $x_h = 1$.

x_{h-2}	u_{h-2}^*	$x_{h-1} = u_{h-2}^*$	u_{h-1}^*
1	2	2	1
2	2	2	1
3	2	2	1

Q4. Optimisation-based Control — (b)

[10 marks]

A water tank has the model $x_{k+1} = x_k + 0.1 u_k - \alpha x_k$. Collecting the state terms:

$$\begin{aligned} x_{k+1} &= x_k + 0.1 u_k - \alpha x_k \\ &= (1 - \alpha)x_k + 0.1 u_k, \end{aligned}$$

where x is the water level, u_k is the input flow at time k , and $\alpha \ll 1$ is the outflow rate. Given $x_0 = 0.9$, $|u_k| \leq 1$ for all k , desired level $\bar{x} = 1.0$, horizon $N = 5$, and small input cost $\epsilon \|u\|_2^2$ with $u = [u_0 \ u_1 \ \dots]^T$.

(i) MPC formulation [6 marks]

Step 1 — predict the state trajectory over the horizon.

Rolling out the linear dynamics k steps from x_0 :

$$x_k = (1 - \alpha)^k x_0 + 0.1 \sum_{j=0}^{k-1} (1 - \alpha)^{k-1-j} u_j, \quad k = 1, \dots, 5.$$

Reason: Each application of $x_{k+1} = (1 - \alpha)x_k + 0.1u_k$ adds one term. Input u_j at step j propagates through $(k - 1 - j)$ further steps, each multiplying by $(1 - \alpha)$, hence the $(k - 1 - j)$ exponent.

Step 2 — write as a matrix equation.

Stack the predictions into $\mathbf{x} = [x_1, \dots, x_5]^T$ and $u = [u_0, \dots, u_4]^T$:

$$\mathbf{x} = \mathbf{A}x_0 + \mathbf{B}u,$$

where $\mathbf{A} = [(1 - \alpha), \dots, (1 - \alpha)^5]^T \in \mathbb{R}^5$ and $\mathbf{B} \in \mathbb{R}^{5 \times 5}$ is the lower-triangular Toeplitz matrix with (i, j) entry $0.1(1 - \alpha)^{i-j-1}$ for $i > j$, and zero for $i \leq j$.

Reason: $\mathbf{B}$ is lower-triangular because u_j can only affect future states x_i with $i > j$. The (i, j) entry captures how many steps u_j propagates to reach x_i .

Step 3 — state the MPC optimisation problem.

$$\begin{aligned} \min_u \quad & \|\mathbf{B}u + \mathbf{A}x_0 - \mathbf{1}\|_2^2 + \epsilon \|u\|_2^2 \\ \text{subject to} \quad & -1 \leq u_k \leq 1, \quad k = 0, 1, 2, 3, 4. \end{aligned}$$

Reason: The term $\mathbf{B}u + \mathbf{A}x_0 - \mathbf{1}$ is the predicted tracking error $\mathbf{x} - \bar{\mathbf{x}}\mathbf{1}$. The input cost $\epsilon \|u\|_2^2$ regularises the problem. No terminal cost is included per the hint; $x_0 = 0.9$ is a fixed parameter.

(ii) Quadratic programme? [2 marks]

Step 1 — expand the objective.

Let $e = \mathbf{B}u + \mathbf{A}x_0 - \mathbf{1}$ (the tracking error vector):

$$\begin{aligned} J(u) &= e^\top e + \epsilon u^\top u \\ &= (\mathbf{B}u + \mathbf{A}x_0 - \mathbf{1})^\top (\mathbf{B}u + \mathbf{A}x_0 - \mathbf{1}) + \epsilon u^\top u \\ &= u^\top \mathbf{B}^\top \mathbf{B} u + 2(\mathbf{A}x_0 - \mathbf{1})^\top \mathbf{B} u + \text{const} + \epsilon u^\top u \\ &= u^\top \underbrace{(\mathbf{B}^\top \mathbf{B} + \epsilon I)}_H u + 2(\mathbf{A}x_0 - \mathbf{1})^\top \mathbf{B} u + \text{const}. \end{aligned}$$

Reason: Expanding $e^\top e = (\mathbf{B}u)^\top (\mathbf{B}u) + 2(\mathbf{A}x_0 - \mathbf{1})^\top \mathbf{B} u + \text{const}$; the cross-term is $2(\mathbf{A}x_0 - \mathbf{1})^\top \mathbf{B} u$ (linear in u).

Step 2 — identify the problem class.

Yes — this is a **convex quadratic programme (QP)**. $J(u)$ is quadratic in u with Hessian $H = \mathbf{B}^\top \mathbf{B} + \epsilon I \succ 0$ (positive definite since $\mathbf{B}^\top \mathbf{B} \succeq 0$ and $\epsilon I \succ 0$). Constraints $-1 \leq u_k \leq 1$ are linear. The problem has a unique global minimiser.

(iii) Achievable performance for $\alpha = 0.12$ [2 marks]

Step 1 — find the steady-state equation.

At steady state with constant u_{ss} , set $x_{k+1} = x_k = x_{ss}$:

$$\begin{aligned} x_{ss} &= (1 - \alpha)x_{ss} + 0.1 u_{ss} \\ x_{ss} - (1 - \alpha)x_{ss} &= 0.1 u_{ss} \\ \alpha x_{ss} &= 0.1 u_{ss} \\ x_{ss} &= \frac{0.1}{\alpha} u_{ss}. \end{aligned}$$

Reason: The steady state is where the system does not change between time steps.

Step 2 — apply the input constraint and substitute $\alpha = 0.12$.

With $|u_{ss}| \leq 1$, the maximum steady-state level is achieved at $u_{ss} = 1$:

$$x_{ss, \max} = \frac{0.1}{0.12} \times 1 = \frac{5}{6} \approx 0.833.$$

The desired level $\bar{x} = 1.0$ is **not achievable**. The input constraint $|u_k| \leq 1$ limits the maximum steady-state water level to $\approx 0.833 < 1.0$. The MPC controller will saturate at $u_k = 1$ throughout the horizon and still fall short of the target.

Q4. Optimisation-based Control — (c)

[2 marks]

Briefly explain the difference between *lbest* and *gbest* particle swarm optimisation. Mention at least one advantage of each.

gbest (global best): Every particle is attracted toward the single best position found by *any* particle in the entire swarm (star topology).

Advantage: Fast convergence — information propagates instantly across the whole swarm, so good solutions are exploited quickly.

lbest (local best): Each particle is attracted only toward the best position found within a small neighbourhood (ring or lattice topology).

Advantage: Better exploration and resistance to premature convergence — isolated sub-swarms can pursue different regions of the search space simultaneously, reducing the risk of getting trapped in a local optimum.

Reason: The key trade-off: *gbest* converges faster but is more susceptible to premature convergence on multi-modal problems; *lbest* is slower but maintains diversity longer.